

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 09, 2025
Milestone 3

Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Milestone3>

Video

<https://www.loom.com/share/8b1ac99b02a140b4a7d6ef4c050dcf95>

UML Class Diagram

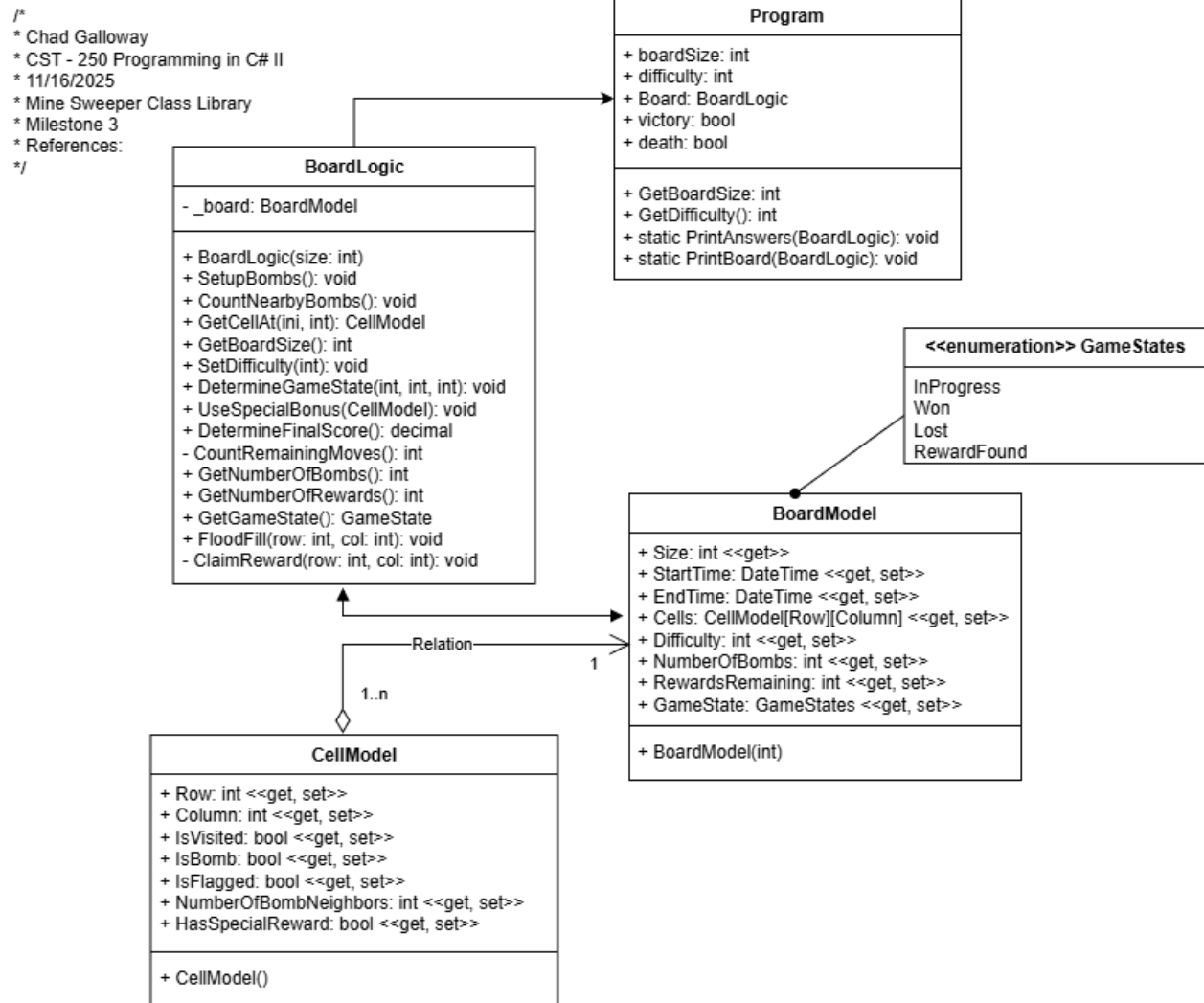


Figure 1: UML Class Diagram with Milestone 3 changes

Screen Shots

```
C:\Users\WarWagon\OneDrive x + v
Welcome to Mine Sweeper Console Application!

Board size: 10x10, Difficulty: Low
Good Luck!!

Here is the answer key for the board.
  1  2  3  4  5  6  7  8  9 10
+---+---+---+---+---+---+---+---+---+
1| . | 1 | B | B | 1 | . | . | . | . |
+---+---+---+---+---+---+---+---+
2| . | 1 | 2 | 3 | 2 | 1 | . | 1 | 1 | 1 |
+---+---+---+---+---+---+---+---+
3| . | . | . | . | 1 | B | 1 | . | 1 | B | 1 |
+---+---+---+---+---+---+---+---+
4| 2 | 2 | 1 | 1 | 1 | 1 | . | 2 | 2 | 2 |
+---+---+---+---+---+---+---+---+
5| B | B | 1 | . | . | . | . | 1 | B | 1 |
+---+---+---+---+---+---+---+---+
6| 2 | 2 | 1 | . | . | . | . | 1 | 1 | 1 |
+---+---+---+---+---+---+---+---+
7| 1 | 1 | . | . | . | . | . | . | . | . |
+---+---+---+---+---+---+---+---+
8| B | 1 | . | . | . | . | . | . | 1 | 1 |
+---+---+---+---+---+---+---+---+
9| 1 | 1 | . | . | . | . | . | 1 | 2 | B |
+---+---+---+---+---+---+---+---+
10| . | . | . | . | . | . | . | 1 | B | 2 |
+---+---+---+---+---+---+---+---+

Here is the current board.
  1  2  3  4  5  6  7  8  9 10
+---+---+---+---+---+---+---+---+---+
1| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
2| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
3| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
4| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
5| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
6| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
7| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
8| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
9| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+
10| ? | ? | ? | ? | ? | ? | ? | ? | ? | ? |
+---+---+---+---+---+---+---+---+---+

There are still 10 bombs that are unaccounted for.
What is your next move.
Enter row (1 - 10): |
```

Figure 2: Screenshot of the initial loading state of the game after selecting 10 for board size and 1 for difficulty

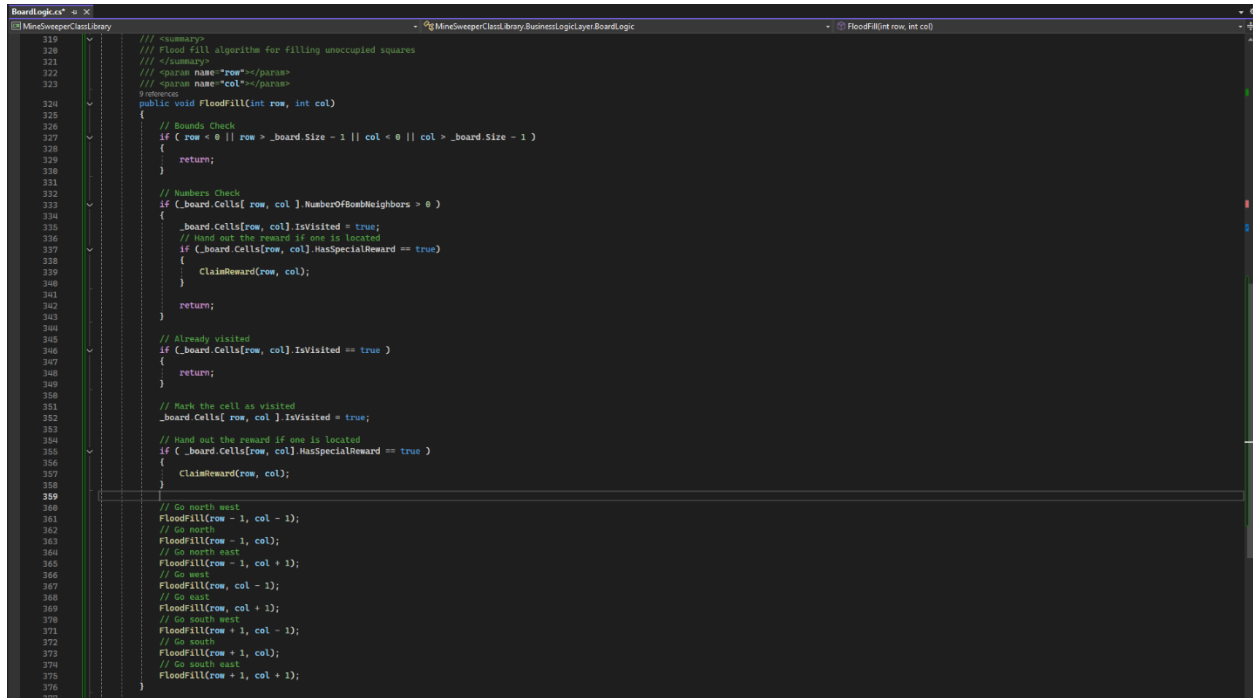
```

C:\Users\WarWagon\OneDrive  X  +  -  x
7| 1 | 1 | . | . | . | . | . | . | . | . |
+-----+
8| B | 1 | . | . | . | . | . | . | 1 | 1 |
+-----+
9| 1 | 1 | . | . | r | . | . | . | 1 | 2 | B |
+-----+
10| . | . | . | . | . | . | . | . | 1 | B | 2 |
+-----+
Here is the current board.
  1  2  3  4  5  6  7  8  9  10
+-----+
1| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
2| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
3| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
4| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
5| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
6| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
7| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
8| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
9| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
10| ? | ? | ? | ? | ? | ? | ? | ? | ? |
+-----+
There are still 10 bombs that are unaccounted for.
What is your next move.
Enter row (1 - 10): 6
Enter column (1 - 10): 6
Enter choice ( 1 = Visit Cell, 2 = Flag Cell ): 1
Here is the current board.
  1  2  3  4  5  6  7  8  9  10
+-----+
1| ? | ? | ? | ? | ? | 1 | . | . | . |
+-----+
2| ? | ? | ? | ? | ? | 2 | 1 | . | 1 | 1 |
+-----+
3| ? | ? | ? | ? | ? | 1 | . | 1 | ? | ? |
+-----+
4| ? | ? | 1 | 1 | 1 | 1 | . | 2 | ? | ? |
+-----+
5| ? | ? | 1 | . | . | . | . | 1 | ? | ? |
+-----+
6| ? | 2 | 1 | . | . | . | . | 1 | 1 | 1 |
+-----+
7| ? | 1 | . | . | . | . | . | . | . |
+-----+
8| ? | 1 | . | . | . | . | . | . | 1 | 1 |
+-----+
9| 1 | 1 | . | . | . | . | . | 1 | 2 | ? |
+-----+
10| . | . | . | . | . | . | . | 1 | ? | ? |
+-----+
** You found a reward ** You have 1 rewards left.
There are still 10 bombs that are unaccounted for.
What is your next move.
Enter row (1 - 10): |

```

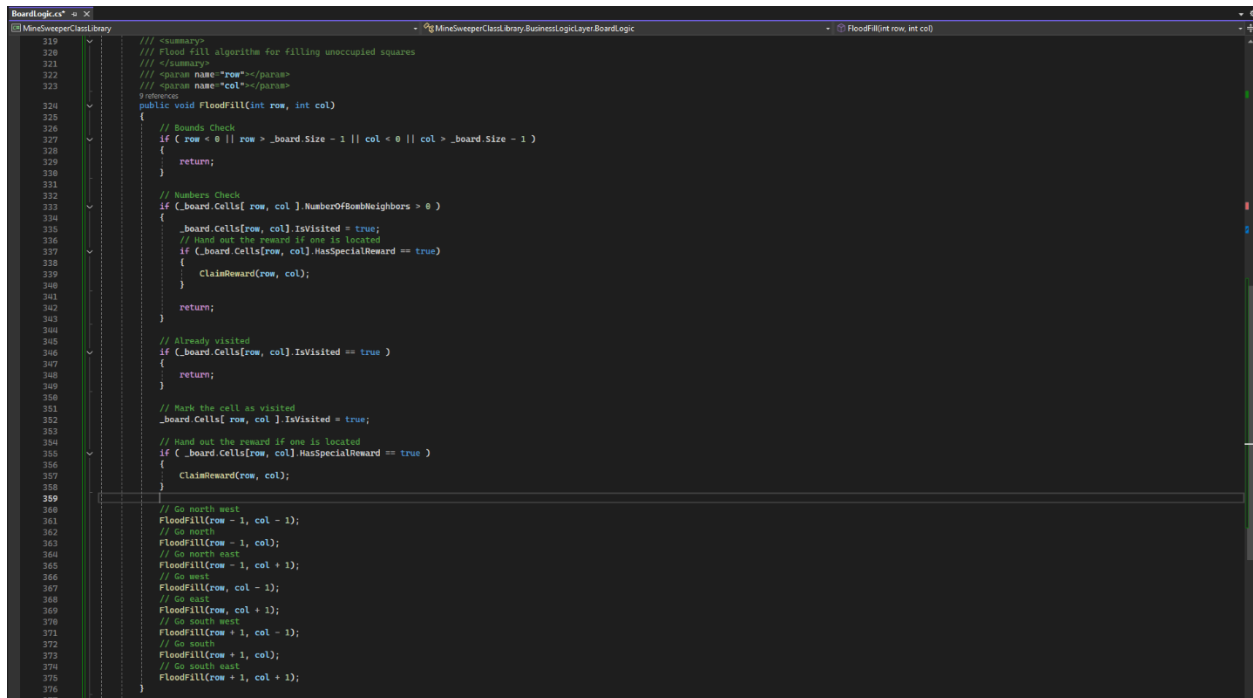
Figure 3: Screenshot of flood fill of an area after selecting 6, 6 to visit. The reward was still collected even if the user did not click directly on it.

Code Changes



```
319 // <summary>
320 // Flood fill algorithm for filling unoccupied squares
321 // </summary>
322 // <param name="row">/param>
323 // <param name="col">/param>
324 // references
325 public void FloodFill(int row, int col)
326 {
327     // Bounds Check
328     if ( row < 0 || row > _board.Size - 1 || col < 0 || col > _board.Size - 1 )
329     {
330         return;
331     }
332     // Numbers Check
333     if ( _board.Cells[ row, col ].NumberOfBombNeighbors > 0 )
334     {
335         _board.Cells[ row, col ].IsVisited = true;
336         // Hand out the reward if one is located
337         if ( _board.Cells[ row, col ].HasSpecialReward == true )
338         {
339             ClaimReward( row, col );
340         }
341         return;
342     }
343     // Already visited
344     if ( _board.Cells[ row, col ].IsVisited == true )
345     {
346         return;
347     }
348     // Mark the cell as visited
349     _board.Cells[ row, col ].IsVisited = true;
350     // Hand out the reward if one is located
351     if ( _board.Cells[ row, col ].HasSpecialReward == true )
352     {
353         ClaimReward( row, col );
354     }
355     // Go north west
356     FloodFill( row - 1, col - 1 );
357     // Go north
358     FloodFill( row - 1, col );
359     // Go north east
360     FloodFill( row - 1, col + 1 );
361     // Go west
362     FloodFill( row, col - 1 );
363     // Go east
364     FloodFill( row, col + 1 );
365     // Go south west
366     FloodFill( row + 1, col - 1 );
367     // Go south
368     FloodFill( row + 1, col );
369     // Go south east
370     FloodFill( row + 1, col + 1 );
371 }
```

Figure 4: Screenshot of flood fill code addition.



```
319 // <summary>
320 // Flood fill algorithm for filling unoccupied squares
321 // </summary>
322 // <param name="row">/param>
323 // <param name="col">/param>
324 // references
325 public void ClaimReward(int row, int col)
326 {
327     // Bounds Check
328     if ( row < 0 || row > _board.Size - 1 || col < 0 || col > _board.Size - 1 )
329     {
330         return;
331     }
332     // Numbers Check
333     if ( _board.Cells[ row, col ].NumberOfBombNeighbors > 0 )
334     {
335         _board.Cells[ row, col ].IsVisited = true;
336         // Hand out the reward if one is located
337         if ( _board.Cells[ row, col ].HasSpecialReward == true )
338         {
339             ClaimReward( row, col );
340         }
341         return;
342     }
343     // Already visited
344     if ( _board.Cells[ row, col ].IsVisited == true )
345     {
346         return;
347     }
348     // Mark the cell as visited
349     _board.Cells[ row, col ].IsVisited = true;
350     // Hand out the reward if one is located
351     if ( _board.Cells[ row, col ].HasSpecialReward == true )
352     {
353         ClaimReward( row, col );
354     }
355     // Go north west
356     ClaimReward( row - 1, col - 1 );
357     // Go north
358     ClaimReward( row - 1, col );
359     // Go north east
360     ClaimReward( row - 1, col + 1 );
361     // Go west
362     ClaimReward( row, col - 1 );
363     // Go east
364     ClaimReward( row, col + 1 );
365     // Go south west
366     ClaimReward( row + 1, col - 1 );
367     // Go south
368     ClaimReward( row + 1, col );
369     // Go south east
370     ClaimReward( row + 1, col + 1 );
371 }
```

Figure 5: Screenshot of ClaimRewards() method all rewards were handled in the determine game state method but I moved it to the flood fill and into its own method because it needed to be called from two different locations to function properly. Ideally, I would like to move it to an

event and add a listener but I need to do more research on that. I fully understand the concept of events and listeners but I am foggy on the implementation so hopefully next week I can have it in to remove the overhead in the flood fill.

Summary of Concepts

This week we learned recursion and how to use it in practical applications. Minesweeper is just one instance of such an occasion using recursion flood fill to fill all the empty space on the map is an excellent demonstration of a typical use case. Other cases involve image editing applications and terrain manipulation algorithms where 3D voids are space might be present. Recursion is a rather elegant way to handle many complex problems by breaking them down into smaller more manageable chunks.